```
# Colab에 Mecab 설치
!pip install konlpy
!pip install mecab-python
!bash <(curl -s https://raw.githubusercontent.com/konlpy/konlpy/master/scripts/mecab.sh)
```



```
mecab-python is already installed
Done.
```

```
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import urllib.request
from collections import Counter
from konlpy.tag import Mecab
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
urllib.request.urlretrieve("https://raw.githubusercontent.com/bab2min/corpus/master/sentiment/naver_sh
```

```
('ratings_total.txt', <http.client.HTTPMessage at 0x7ce649408580>)
```

```
total_data = pd.read_table('ratings_total.txt', names=['ratings', 'reviews'])
print('전체 리뷰 개수 :',len(total_data)) # 전체 리뷰 개수 출력
```

```
전체 리뷰 개수 : 200000
```

```
total_data[:5]
```

| | ratings | reviews |
|---|---|---|
| **0** | 5 | 배공빠르고 굿 |
| **1** | 2 | 택배가 엉망이네용 저희집 밑에층에 말도없이 놔두고가고 |
| **2** | 5 | 아주좋아요 바지 정말 좋아서2개 더 구매했어요 이가격에 대박입니다. 바느질이 조금 ... |
| **3** | 2 | 선물용으로 빨리 받아서 전달했어야 하는 상품이었는데 머그컵만 와서 당황했습니다. 전... |

```
total_data['label'] = np.select([total_data.ratings > 3], [1], default=0)
total_data[:5]
```

| | ratings | reviews | label |
|---|---|---|---|
| **0** | 5 | 배공빠르고 굿 | 1 |
| **1** | 2 | 택배가 엉망이네용 저희집 밑에층에 말도없이 놔두고가고 | 0 |
| **2** | 5 | 아주좋아요 바지 정말 좋아서2개 더 구매했어요 이가격에 대박입니다. 바느질이 조금 ... | 1 |
| **3** | 2 | 선물용으로 빨리 받아서 전달했어야 하는 상품이었는데 머그컵만 와서 당황했습니다. 전... | 0 |
| **4** | 5 | 민트색상 예뻐요. 옆 손잡이는 거는 용도로도 사용되네요 ㅎㅎ | 1 |

```
total_data['ratings'].nunique(), total_data['reviews'].nunique(), total_data['label'].nunique()
```

```
(4, 199908, 2)
```

```
total_data.drop_duplicates(subset=['reviews'], inplace=True) # reviews 열에서 중복인 내용이 있다면 중복
print('총 샘플의 수 :',len(total_data))
```

```
총 샘플의 수 : 199908
```

```
print(total_data.isnull().values.any())
```

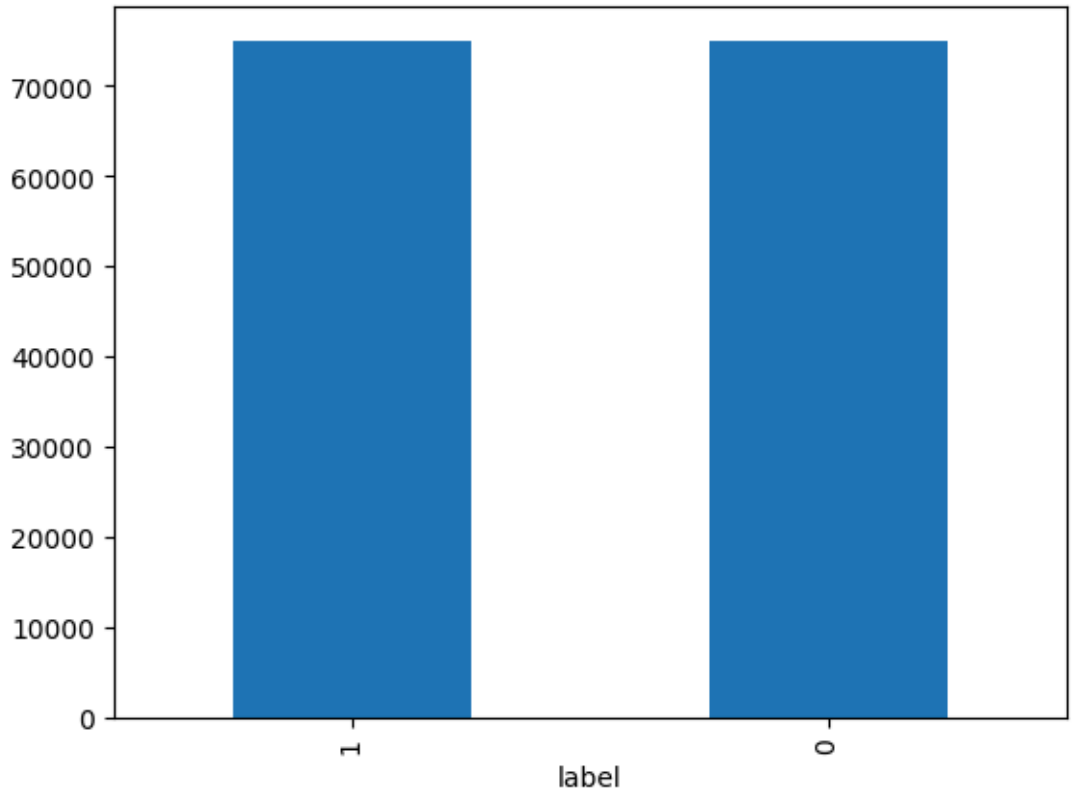
```
False
```

```
train_data, test_data = train_test_split(total_data, test_size = 0.25, random_state = 42)
print('훈련용 리뷰의 개수 :', len(train_data))
print('테스트용 리뷰의 개수 :', len(test_data))
```

```
훈련용 리뷰의 개수 : 149931
테스트용 리뷰의 개수 : 49977
```

```
#레이블의 분포 확인
train_data['label'].value_counts().plot(kind = 'bar')
```

```
<Axes: xlabel='label'>
```




```
print(train_data.groupby('label').size().reset_index(name = 'count'))
```

```
   label  count
0      0  74918
1      1  75013
```

```
#데이터 정제하기
# 한글과 공백을 제외하고 모두 제거
train_data['reviews'] = train_data['reviews'].str.replace("[^ㄱ-ㅎㅏ-ㅣ가-힣 ]","")
train_data['reviews'].replace('', np.nan, inplace=True)
print(train_data.isnull().sum())
```

```
ratings    0
reviews    0
label      0
dtype: int64
```

```
test_data.drop_duplicates(subset = ['reviews'], inplace=True) # 중복 제거
test_data['reviews'] = test_data['reviews'].str.replace("[^ㄱ-ㅎㅏ-ㅣ가-힣 ]","") # 정규 표현식 수행
test_data['reviews'].replace('', np.nan, inplace=True) # 공백은 Null 값으로 변경
test_data = test_data.dropna(how='any') # Null 값 제거
print('전처리 후 테스트용 샘플의 개수 :',len(test_data))
```

```
전처리 후 테스트용 샘플의 개수 : 49977
```

```
#토큰화
mecab = Mecab()
print(mecab.morphs('와 이런 것도 상품이라고 차라리 내가 만드는 게 나을 뻔'))
```

```
['와', '이런', '것', '도', '상품', '이', '라고', '차라리', '내', '가', '만드', '는', '게', '나을',
```

```
#불용어 지정하여 필요없는 토큰들은 제거
stopwords = ['도', '는', '다', '의', '가', '이', '은', '한', '에', '하', '고', '을', '를', '인', '듯',
```

```
train_data['tokenized'] = train_data['reviews'].apply(mecab.morphs)
train_data['tokenized'] = train_data['tokenized'].apply(lambda x: [item for item in x if item not in s
```

```
test_data['tokenized'] = test_data['reviews'].apply(mecab.morphs)
test_data['tokenized'] = test_data['tokenized'].apply(lambda x: [item for item in x if item not in sto
```

### 6) 단어와 길이 분포 확인하기

긍정 리뷰에는 주로 어떤 단어들이 많이 등장하고, 부정 리뷰에는 주로 어떤 단어들이 등장하는지 두 가지 경우에 대해서 각 단어의 빈도수를 계산해보겠습니다. 각 레이블에 따라서 별도로 단어들의 리스트를 저장해줍니다.

```
negative_words = np.hstack(train_data[train_data.label == 0]['tokenized'].values)
positive_words = np.hstack(train_data[train_data.label == 1]['tokenized'].values)
```

```
#counter()를 사용하여 각 단어의 빈도수 카운트
negative_word_count = Counter(negative_words)
print(negative_word_count.most_common(20))
```

```
[('.', 77185), ('네요', 31942), ('는데', 20241), ('안', 19757), ('어요', 15232), ('있', 13203), (
```

```
positive_word_count = Counter(positive_words)
print(positive_word_count.most_common(20))
```

```
[('좋', 39455), ('.', 35659), ('아요', 21278), ('네요', 19912), ('어요', 19359), ('잘', 18608), (
```

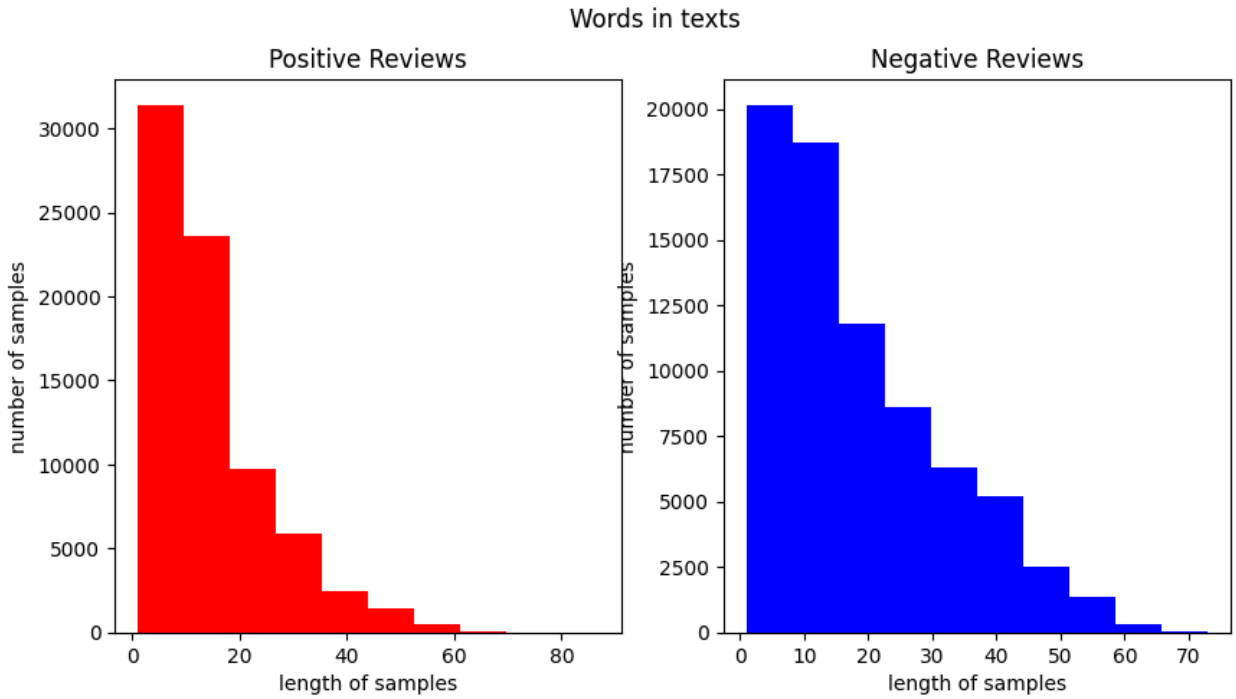
'네요','는데','안','않','너무','없' 등과 같은 단어들이 부정 리뷰에서 주로 등장합니다. 긍정 리뷰에 대해서도 출력해봅시다.

'좋','아요','네요','잘','너무','만족' 등과 같은 단어들이 주로 많이 등장합니다. 두 가지 경우에 대해서 각각 길이 분포를 확인해봅시다.

```
fig,(ax1,ax2) = plt.subplots(1,2,figsize=(10,5))
text_len = train_data[train_data['label']==1]['tokenized'].map(lambda x: len(x))
ax1.hist(text_len, color='red')
ax1.set_title('Positive Reviews')
ax1.set_xlabel('length of samples')
ax1.set_ylabel('number of samples')
print('긍정 리뷰의 평균 길이 :', np.mean(text_len))

text_len = train_data[train_data['label']==0]['tokenized'].map(lambda x: len(x))
ax2.hist(text_len, color='blue')
ax2.set_title('Negative Reviews')
fig.suptitle('Words in texts')
ax2.set_xlabel('length of samples')
ax2.set_ylabel('number of samples')
print('부정 리뷰의 평균 길이 :', np.mean(text_len))
plt.show()
```

```
긍정 리뷰의 평균 길이 : 14.788583312225882
부정 리뷰의 평균 길이 : 18.835420059264795
```




```
X_train = train_data['tokenized'].values
y_train = train_data['label'].values
X_test= test_data['tokenized'].values
y_test = test_data['label'].values
```

### 7) 정수 인코딩

기계가 텍스트를 숫자로 처리할 수 있도록 훈련 데이터와 테스트 데이터에 정수 인코딩을 수행해야 합니다. 훈련 데이터에 대해서 단어 집합(vocaburary)을 만들어봅시다!

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(X_train)
```

단어 집합이 생성되는 동시에 각 단어에 고유한 정수가 부여되었습니다. 이는 tokenizer.word_index를 출력하여 확인 가능합니다. 등장 횟수가 1회인 단어들은 자연어 처리에서 배제하고자 합니다. 이 단어들이 이 데이터에서 얼만큼의 비중을 차지하는지 확인해봅시다.

```
threshold = 2
total_cnt = len(tokenizer.word_index) # 단어의 수
rare_cnt = 0 # 등장 빈도수가 threshold보다 작은 단어의 개수를 카운트
total_freq = 0 # 훈련 데이터의 전체 단어 빈도수 총 합
rare_freq = 0 # 등장 빈도수가 threshold보다 작은 단어의 등장 빈도수의 총 합

# 단어와 빈도수의 쌍(pair)을 key와 value로 받는다.
for key, value in tokenizer.word_counts.items():
    total_freq = total_freq + value

    # 단어의 등장 빈도수가 threshold보다 작으면
    if(value < threshold):
        rare_cnt = rare_cnt + 1
        rare_freq = rare_freq + value

print('단어 집합(vocabulary)의 크기 :',total_cnt)
print('등장 빈도가 %s번 이하인 희귀 단어의 수: %s'%(threshold - 1, rare_cnt))
print("단어 집합에서 희귀 단어의 비율:", (rare_cnt / total_cnt)*100)
print("전체 등장 빈도에서 희귀 단어 등장 빈도 비율:", (rare_freq / total_freq)*100)
```

```
단어 집합(vocabulary)의 크기 : 42360
등장 빈도가 1번 이하인 희귀 단어의 수: 19706
단어 집합에서 희귀 단어의 비율: 46.520302171860244
전체 등장 빈도에서 희귀 단어 등장 빈도 비율: 0.7818451322939414
```

단어가 약 40000개 존재.. 등장 횟수가 1회인 단어들은 45% 차지.. 하지만 실제로 훌련 데이터에서 등장 빈도로 차지하는 비중은 매우 적은 수치인 약 0.8%밖에 되지 않는다. 그래서 등장 빈도수가 1인 단어들의 수를 제외한 단어의 개수를 단어 집합의 최대 크기로 제한한다.

```
# 전체 단어 개수 중 빈도수 2이하인 단어 개수는 제거.
# 0번 패딩 토큰과 1번 OOV 토큰을 고려하여 +2
vocab_size = total_cnt - rare_cnt + 2
print('단어 집합의 크기 :',vocab_size)
```

```
단어 집합의 크기 : 22656
```

```
tokenizer = Tokenizer(vocab_size, oov_token = 'OOV')
tokenizer.fit_on_texts(X_train)
X_train = tokenizer.texts_to_sequences(X_train)
X_test = tokenizer.texts_to_sequences(X_test)
```

```
print(X_train[:3])
```

```
[[72, 146, 2083, 313, 14804, 278, 79, 7, 254, 182, 148, 835, 3046, 659, 3, 85, 68, 219, 44, 1402,
```

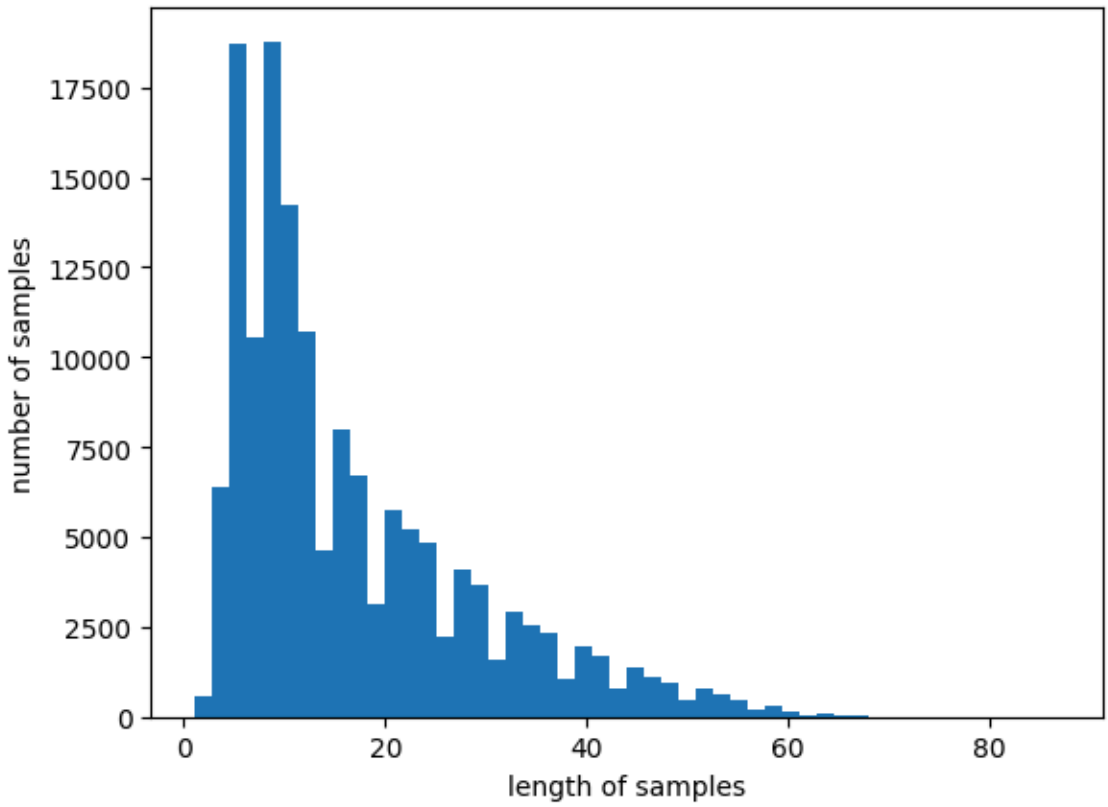
```
print(X_train[:3])
```

```
[[72, 146, 2083, 313, 14804, 278, 79, 7, 254, 182, 148, 835, 3046, 659, 3, 85, 68, 219, 44, 1402,
```

## 8) 패딩

```
print('리뷰의 최대 길이 :',max(len(review) for review in X_train))
print('리뷰의 평균 길이 :',sum(map(len, X_train))/len(X_train))
plt.hist([len(review) for review in X_train], bins=50)
plt.xlabel('length of samples')
plt.ylabel('number of samples')
plt.show()
```

```
리뷰의 최대 길이 : 87
리뷰의 평균 길이 : 16.8107195976816
```



```
def below_threshold_len(max_len, nested_list):
  count = 0
  for sentence in nested_list:
    if(len(sentence) <= max_len):
        count = count + 1
  print('전체 샘플 중 길이가 %s 이하인 샘플의 비율: %s'%(max_len, (count / len(nested_list))*100))



max_len = 80
below_threshold_len(max_len, X_train)
```

```
전체 샘플 중 길이가 80 이하인 샘플의 비율: 99.99866605305107
```

```
X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)
```

## 3. GRU로 네이버 쇼핑 리뷰 감성 분류하기

하이퍼파라미터인 임베딩 벡터의 차원은 100, 은닉 상태의 크기는 128입니다. 모델은 다 대 일 구조의 LSTM을 사용합니다. 해당 모델은 마지막 시점에서 두 개의 선택지 중 하나를 예측하는 이진 분류 문제를 수행하는 모델입니다. 이진 분류 문제의 경우 출력층에 로지스틱 회귀를 사용해야 하므로 활성화 함수로는 시그모이드 함수를 사용하고, ,손실 함수로 크로스 엔트로피 함수를 사용합니다. 하이퍼파라미터인 배치 크기는 64이며, 15 에포크를 수행합니다.

EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=4)는 검증 데이터 손실(val_loss)이 증가하면, 과적합 징후므로 검증 데이터 손실이 4회 증가하면 정해진 에포크가 도달하지 못하였더라도 학습을 조기 종료(Early Stopping)한다는 의미입니다. ModelCheckpoint를 사용하여 검증 데이터의 정확도(val_acc)가 이전보다 좋아질 경우에만 모델을 저장합니다. validation_split=0.2을 사용하여 훈련 데이터의 20%를 검증 데이터로 분리해서 사용하고, 검증 데이터를 통해서 훈련이 적절히 되고 있는지 확인합니다. 검증 데이터는 기계가 훈련 데이터에 과적합되고 있지는 않은지 확인하기 위한 용도로 사용됩니다.

```
from tensorflow.keras.layers import Embedding, Dense, GRU
from tensorflow.keras.models import Sequential
from tensorflow.keras.models import load_model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

embedding_dim = 100
hidden_units = 128

model = Sequential()
model.add(Embedding(vocab_size, embedding_dim))
model.add(GRU(hidden_units))
model.add(Dense(1, activation='sigmoid'))

es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=4)
mc = ModelCheckpoint('best_model.h5', monitor='val_acc', mode='max', verbose=1, save_best_only=True)
```

```
model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['acc'])
history = model.fit(X_train, y_train, epochs=15, callbacks=[es, mc], batch_size=64, validation_split=0.2
```

```
Epoch 1/15
1875/1875 [==============================] - ETA: 0s - loss: 0.2881 - acc: 0.8898
Epoch 1: val_acc improved from -inf to 0.91250, saving model to best_model.h5
1875/1875 [==============================] - 415s 220ms/step - loss: 0.2881 - acc: 0.8898 - val_lo
Epoch 2/15
/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103: UserWarning: You are sa
  saving_api.save_model(
1875/1875 [==============================] - ETA: 0s - loss: 0.2260 - acc: 0.9193
Epoch 2: val_acc improved from 0.91250 to 0.92023, saving model to best_model.h5
1875/1875 [==============================] - 375s 200ms/step - loss: 0.2260 - acc: 0.9193 - val_lo
Epoch 3/15
1875/1875 [==============================] - ETA: 0s - loss: 0.2039 - acc: 0.9279
Epoch 3: val_acc improved from 0.92023 to 0.92450, saving model to best_model.h5
1875/1875 [==============================] - 377s 201ms/step - loss: 0.2039 - acc: 0.9279 - val_lo
Epoch 4/15
1875/1875 [==============================] - ETA: 0s - loss: 0.1895 - acc: 0.9333
Epoch 4: val_acc did not improve from 0.92450
1875/1875 [==============================] - 374s 200ms/step - loss: 0.1895 - acc: 0.9333 - val_lo
Epoch 5/15
1875/1875 [==============================] - ETA: 0s - loss: 0.1780 - acc: 0.9387
Epoch 5: val_acc improved from 0.92450 to 0.92560, saving model to best_model.h5
1875/1875 [==============================] - 375s 200ms/step - loss: 0.1780 - acc: 0.9387 - val_lo
Epoch 6/15
1875/1875 [==============================] - ETA: 0s - loss: 0.1679 - acc: 0.9433
Epoch 6: val_acc did not improve from 0.92560
1875/1875 [==============================] - 382s 204ms/step - loss: 0.1679 - acc: 0.9433 - val_lo
Epoch 7/15
1875/1875 [==============================] - ETA: 0s - loss: 0.1580 - acc: 0.9470
Epoch 7: val_acc did not improve from 0.92560
1875/1875 [==============================] - 376s 201ms/step - loss: 0.1580 - acc: 0.9470 - val_lo
Epoch 8/15
1875/1875 [==============================] - ETA: 0s - loss: 0.1491 - acc: 0.9507
Epoch 8: val_acc did not improve from 0.92560
1875/1875 [==============================] - 374s 199ms/step - loss: 0.1491 - acc: 0.9507 - val_lo
Epoch 9/15
1875/1875 [==============================] - ETA: 0s - loss: 0.1399 - acc: 0.9543
Epoch 9: val_acc did not improve from 0.92560
1875/1875 [==============================] - 406s 217ms/step - loss: 0.1399 - acc: 0.9543 - val_lo
Epoch 9: early stopping
```

## 4. 리뷰 예측해보기

임의의 문장에 대한 예측을 위해서는 학습하기 전 전처리를 동일하게 적용해줍니다. 전처리의 순서는 정규 표현식을 통한 한국어 외 문자 제거, 토큰화, 불용어 제거, 정수 인코딩, 패딩 순입니다.

```
def sentiment_predict(new_sentence):
  new_sentence = re.sub(r'[^ㄱ-ㅎㅏ-ㅣ가-힣 ]','', new_sentence)
  new_sentence = mecab.morphs(new_sentence)
  new_sentence = [word for word in new_sentence if not word in stopwords]
  encoded = tokenizer.texts_to_sequences([new_sentence])
  pad_new = pad_sequences(encoded, maxlen = max_len)

sentiment_predict('이 상품 진짜 좋아요... 저는 강추합니다. 대박')
```

```
1/1 [==============================] - 1s 659ms/step
97.16% 확률로 긍정 리뷰입니다.
<ipython-input-40-eadb03d4eaac>:8: DeprecationWarning: Conversion of an array with ndim > 0 to a s
  score = float(model.predict(pad_new))
```

```
#우리 csv 파일에서 reviews_text만 분석하게끔 입력
import pandas as pd

def sentiment_predict_from_csv(reviews_style1, column_name):
    # Load the CSV file
    df = pd.read_csv(reviews_style1)

    # Iterate over each row in the DataFrame
    for index, row in df.iterrows():
        # Extract the text from the specified column
        new_sentence = row[column_name]

        # Perform sentiment prediction
        new_sentence = re.sub(r'[^ㄱ-ㅎㅏ-ㅣ가-힣 ]', '', new_sentence)
        new_sentence = mecab.morphs(new_sentence)
        new_sentence = [word for word in new_sentence if not word in stopwords]
        encoded = tokenizer.texts_to_sequences([new_sentence])
        pad_new = pad_sequences(encoded, maxlen=max_len)

        score = float(model.predict(pad_new))
        if score > 0.5:
            print("Row {}: {:.2f}% 확률로 긍정 리뷰입니다.".format(index, score * 100))
        else:
            print("Row {}: {:.2f}% 확률로 부정 리뷰입니다.".format(index, (1 - score) * 100))

# Example usage:
sentiment_predict_from_csv('reviews_style1.csv', 'review_text')
```

```
1/1 [==============================] - 0s 29ms/step
Row 0: 73.51% 확률로 부정 리뷰입니다.
1/1 [==============================] - 0s 27ms/step
Row 1: 55.87% 확률로 부정 리뷰입니다.
1/1 [==============================] - 0s 27ms/step
Row 2: 98.76% 확률로 긍정 리뷰입니다.
<ipython-input-49-9a07d3d9a561>:20: DeprecationWarning: Conversion of an array with ndim > 0 to a
  score = float(model.predict(pad_new))
```